

Logan Venter

Senior AI/Platform Engineer

logan.venter@outlook.com | +27 60 974 2113 / +27 76 414 0083 | loganventer.com

Logan Venter is a Senior AI/Platform Engineer with over 20 years of software development experience, based in South Africa. He architects and builds production AI platforms, from agent frameworks and MCP servers to agentic chatbot platforms, multi-agent systems and RAG-powered knowledge bases. He is the author of *tawk*, an open source WhatsApp client for the terminal, and *tawk-mcp*, its MCP server. His current focus is enabling organizations to deploy LLM-powered systems with enterprise-grade reliability.

Professional Experience

Senior AI/Platform Engineer

Telesure Investment Holdings *Aug 2025 - Present*

Permanent employee of Telesure since 1 May 2026. Started as a contractor placed by DotCom Software Solutions, from Aug 2025 to Apr 2026

- Architected and built an agentic AI framework of 18 Python packages and 17 .NET projects, applying iDesign methodology with strict dependency direction and protocol-based dependency inversion
- Exposed the Python framework to .NET through a gRPC sidecar with typed client libraries, so behaviour is written once
- Designed and built production RAG pipelines with hybrid retrieval, hierarchical summarization, and hallucination detection
- Implemented agentic planning and orchestration patterns including ReAct, Plan-Execute, and Adaptive strategy selection
- Applied enterprise security patterns: multi-turn attack detection, prompt injection defense, risk-based capability restriction
- Integrated multiple LLM and embedding providers with factory-based routing and graceful degradation
- Built a multi-bot chatbot platform: a FastAPI service with more than 100 endpoints, a React chat widget library, and an embeddable assistant
- Built a self-service administration portal for bot lifecycle management, knowledge versioning, and deployment orchestration
- Built multi-agent systems on the A2A protocol: code and documentation review agents, an agent that builds agents, a planner and proof gate, and a use-case triage and SDLC document pipeline
- Built a terminal AI coding assistant with permission-gated tools, persistent memory, and a VS Code extension host
- Built a suite of MCP servers for Azure DevOps, Microsoft Teams, Microsoft To Do, Databricks, Toggl Track and web research, with human approval for writes
- Authored an MCP server framework for .NET (now discontinued), then led its retirement in favour of the official C# MCP SDK and migrated the servers to .NET 10
- Designed and built a governed AI gateway on .NET 10 with per-application permissions, budgets, usage metering and audit
- Built shared Microsoft Entra authentication libraries with ten credential strategies and a coalescing token refresh service
- Architected and designed an agentic knowledge ingestion and curation pipeline: an orchestrator and four single-purpose agents keep assistant knowledge bases current, with a fail-closed sensitive-data gate and human review through pull requests
- Ran an automated CI/CD control review across more than 1,000 repositories against 17 fixed checks
- Designed observability infrastructure with structured logging, correlation IDs, and A/B testing metrics

- Wrote the developer onboarding course for the framework: an eight-chapter guide with test-graded exercises
- Leading technical architecture decisions and mentoring team members on AI/ML and clean architecture best practices

Senior Software Developer

Derivco *Nov 2018 - Jun 2025*

- Led internal AI initiative with dynamic prompt generation for OpenAI API integration
- Architected LLM-powered automation tools for SDET and code review workflows
- Applied iDesign architectural methodology for maintainable backend systems
- Led technical development, release, and maintenance of end-to-end software systems in the online gaming industry
- Drove CI/CD pipelines (TeamCity, Octopus Deploy) and technical roadmap for multiple products
- Assessed complexity, scope, risk, and engineering effort for significant software development projects
- Contributed to architecture roadmaps, ensuring adherence to iDesign decomposition and layering principles
- Mentored junior developers

References: Chris Coetzee (+27 82 548 8817), Ockert Du Preez (+27 72 829 0170), Paul Kotze (+27 82 901 7953), Albie van Zyl (+27 79 492 5393), Gopi Kistasami (+27 83 616 1371)

Senior Software Developer GMIC.Net (Consultant)

Dynamic Visual Technologies (DVT) *May 2016 - Nov 2018*

- Developed features and bug fixes on the GMIC.Net enterprise platform serving the mining and minerals industry
- Managed CI/CD build processes on client-hosted infrastructure
- Handled branch and release management with Git using feature, release, and hotfix branching strategy
- Executed system integrations between GMIC.Net and third-party systems at client sites
- Participated in technical discussions and design sessions relating to product development

References: Khumo Jele (+27 81 447 9732)

Software Developer / Team Lead

Seecrypt *Jun 2015 - Apr 2016*

- Windows team lead for the SC3 encrypted communications application (voice, messaging, file transfer)
- Development in C# and C++ with WPF desktop UI and Silverlight cross-platform client
- Implemented encrypted local storage using SQLite-SQLCIPHER with AES-256 encryption
- Managed CI/CD with Bamboo build server and Stash (Bitbucket) for Git hosting, releasing to Windows Store
- Led the Windows team coordinating with scrum master, product owner, and QA

References: Annalinde Singh (+27 76 640 4766), Dino Fizzotti (+44 734 041 5199)

Director

Covariant Consulting *Jul 2014 - May 2015*

- UDCS system design and implementation for dealer network management
- Integration of UDCS front-ends and back-ends into Syspro ERP for finance, inventory, and reporting
- Business operations and client management

References: Ronete McKenzie (+27 61 682 2350)

Software Developer

UD Trucks *Nov 2012 - Jun 2014*

- Designed and implemented the UDCS system architecture for dealer customer service operations
- Integrated UDCS with Syspro ERP for finance, inventory, and reporting

- Full-stack implementation: database, data access, business logic, and front-end
- Managed deployment and verification after QA sign-off, maintained software and provided dealer support

References: Ronete McKenzie (+27 61 682 2350)

Software Developer

Enermatics *Apr 2011 - Oct 2012*

- Designed and implemented the Wattkeeper energy management system for monitoring electrical consumption
- Implemented device communication protocols for energy monitoring hardware (serial and TCP/IP)
- Managed deployment and verification using Hudson build server (CI/CD)
- Assisted with client queries and billing cycles

References: Melissa Wittstock (+27 82 750 0065)

Software Developer

Infotech *Sep 2009 - Mar 2011*

- Initial development of the Wattkeeper energy management platform (later continued at Enermatics)
- Implemented device communication protocols for energy monitoring hardware
- Managed deployment and verification using Hudson build server

References: Melissa Wittstock (+27 82 750 0065)

Programmer

Information Trade World *Jul 2007 - Aug 2009*

- Desktop application development in Visual Basic and Delphi with database-driven business applications
- Mainframe COBOL development: batch processing, report generation, and data transformation
- Website development and client system management

References: Maria Jacobs (+27 74 254 6027)

Featured Projects

tawk: WhatsApp in the Terminal

github.com/loganventer/tawk

An open source WhatsApp client for the terminal, written in C with ncurses. It links to a phone the same way WhatsApp Web does, keeps chats in a local SQLite database, and brings notifications, voice notes, media, statuses and themes to the terminal. Independent project, not affiliated with WhatsApp or Meta. MIT licensed.

- Two interchangeable WhatsApp backends behind one contract: whatsmeow linked into the binary, or Baileys in a Node.js sidecar, switched with one config key
- Up to eight WhatsApp accounts in one window and one database, with merged conversations and a sending number per contact
- Photos, videos and PDF pages drawn as real pixels on Sixel terminals, with a block-character fallback elsewhere
- Voice notes, statuses, scheduled messages, link previews, search, reactions, replies and edits
- Chat database encrypted at rest with a passphrase (SQLCipher), plus encrypted backup and restore
- A control socket that lets scripts and AI agents reach a running tawk, with a separate access level per account
- 60 themes, a settings panel, a setup doctor, and a one-line installer for Linux, macOS and Windows (WSL)
- More than 30 test suites covering storage, migrations, the control protocol, connection recovery and the UI widgets

[C](#) [ncurses](#) [SQLite](#) [SQLCipher](#) [Go](#) [Node.js](#) [Sixel](#)

tawk-mcp: MCP Server for tawk

github.com/loganventer/tawk-mcp

An open source Model Context Protocol server that lets AI clients such as Claude Code, Claude Desktop and VS Code read WhatsApp chats through a running tawk and act on them with the owner's approval. Every send waits for approval in tawk, and anything destructive needs two separate confirmations, one of which the model can never give. MIT licensed.

- Close to 90 tools across chats, messages, scheduling, statuses, profile, settings, voice notes and memory
- Six iDesign layers (Host, Clients, Managers, Engines, Resource Access, Core) with every interface bound in one composition root
- Human-in-the-loop by design: approved sends, two-step confirmation for destructive operations, and separate access levels for reading, sending and managing
- Prompt injection defence: message text reaches the model inside clearly marked blocks it is told not to obey
- Memory for writing like the owner: voices tuned per audience with a scorer, contact profiles with a source and confidence on every field, and reply templates
- Knowledge held in the Open Knowledge Format, with optional sync between machines through a private git repository
- Streamable HTTP on loopback with a bearer token by default, stdio when a client should start it, and a Docker image
- Resilient connection to tawk: starts without it, reconnects after restarts, with backoff and a circuit breaker
- Nullable reference types, analysers and warnings as errors on every project

[.NET 10](#) [C#](#) [MCP C# SDK](#) [ASP.NET Core](#) [SQLite](#) [Docker](#)

matrix-clock: Terminal Clock in Digital Rain

A terminal mashup of cmatrix and tty-clock: the Matrix digital rain falls down the screen and renders a large digital clock, either punched out of the rain as a silhouette or filled with falling code. A task panel above the clock shows the running task from Toggl Track and Azure DevOps.

- Two render modes: rain as the background with the clock as a traced silhouette, or rain falling inside the digits
- Four clock styles, seven rain colours, adjustable size and speed, and a justified date line
- Live task lines from Toggl Track and Azure DevOps, with a full-screen browser of the day's entries
- A stats overlay with frame time, frames per second and CPU usage that steps around the clock
- Settings remembered between runs in a plain-text config file
- Builds from C, and from the assembly sources emitted from it, with packaging for Debian, Fedora, AUR and Homebrew

[C](#) [ncurses](#) [libcurl](#) [Assembly](#)

Agentic AI Framework for Python and .NET

A layered toolkit for building LLM agent applications. It covers the agent tool loop, planning, the RAG pipeline, LLM and embedding gateways, prompt injection and agent security, authentication and RBAC, context management, MCP and A2A interop, logging and observability. Python is the reference implementation, and a gRPC sidecar exposes it to .NET through thin typed clients so behaviour is written once.

- 18 independently versioned Python packages and 17 .NET projects, following iDesign layering with protocol-based dependency inversion
- A self-contained gRPC sidecar built for Linux, Windows and macOS and shipped inside the NuGet package, so .NET consumers need no Python install
- Production RAG engine: 14 chunking strategies, 7 vector stores, hybrid retrieval (BM25 + vector), fusion by RRF, CombSUM and weighted scoring, reranking by cross-encoder, ColBERT and LLM, RAPTOR, HyDE, graph retrieval and deep search
- Versioned knowledge bases with commit, activation and rollback
- Planning with 3 strategies (ReAct, Plan-Execute, Adaptive) and a full tool-calling loop with hooks and streaming

- Security pipeline: 43 registered injection patterns, spotlighting, sandwich defence, canary tokens, multi-turn attack detection, tool chain validation and risk-based capability restriction
- 6 LLM gateways and 3 embedding gateways behind one interface, with capability registries for structured output
- Reasoning extensions: Tree of Thoughts, tournament reasoning, Reflexion with procedural memory, evaluator-optimiser loops and human approval gates
- Coverage gates configured per package, at 95% for most, with lint, type and security checks in CI
- A developer onboarding course for the framework: an eight-chapter guide with test-graded exercises and one CI gate per exercise

Python LangChain LangGraph gRPC .NET 10 C# MCP A2A

Enterprise Agentic Chatbot Platform

A multi-bot chat platform built on the agent framework: a FastAPI service with more than 100 endpoints, a themeable React chat widget library, and an embeddable assistant that drops into a host site with one script tag. Features RAG with grounding guards, streaming, voice, knowledge base versioning and A/B testing.

- Single FastAPI service with 102 endpoints covering chat orchestration and bot administration
- 5 RAG strategies (Always, Never, Adaptive, Threshold, Agent) with hybrid search and 3 fusion methods
- A per-bot grounding guard that replaces answers below a confidence tier with a refusal
- Per-conversation context files held in an in-memory vector store
- SSE streaming, real-time audio transcription over WebSocket, and sentence-by-sentence streaming speech output
- Content-addressable knowledge versioning with rollback, and bot configuration kept as code in Git
- React chat widget library with 4 layouts, light and dark themes, 3 transports and WCAG 2.1 AA accessibility
- Embeddable assistant: iframe isolation, an origin-validated postMessage bridge, host theme adoption and single sign-on through a backend-for-frontend session
- Multi-provider auth (Azure AD, JWT, API key) with RBAC, and OpenTelemetry observability with correlation IDs

Python FastAPI LangChain LangGraph React TypeScript Vite SQL Server

Enterprise Agentic Administration Portal

Admin portal for creating, configuring, and managing chatbot agents. Features a 9-tab bot creation wizard with auto-metadata generation, knowledge base versioning with rollback, agent deployment management, a conversation audit log and a monitoring dashboard.

- 9-tab bot creation wizard with schema validation on every step and SSE progress streaming
- LLM-powered auto-generation of bot personality, system prompt, and greetings
- Knowledge base versioning with incremental updates (delta detection, embedding reuse) and rollback
- Conversation audit log showing the thought process and tool calls behind each message
- Agent deployment management with health monitoring
- In-browser chat playground for testing deployed agents
- MVVM layering (ViewModels, Managers, Engines, Accessors) across 11 pages

React TypeScript Vite Zod Python FastAPI

MCP Server Framework for .NET (Discontinued)

Discontinued: the official C# MCP SDK, maintained in collaboration with Microsoft, has replaced this framework. It was a Model Context Protocol server framework for .NET, built before an official SDK matured. It supported multiple transports with session management, JWT and API key authentication, JSON schema validation, and attribute-based tool discovery, and was adopted by other engineers to build servers over line-of-business systems. Logan later led its retirement in favour of the official C# MCP SDK and migrated the servers built on it.

- 5 transports: stdio, REST, Streamable HTTP, SignalR and gRPC

- Attribute-based tool and resource registration with JSON Schema validation of tool inputs
- Per-request API key and JWT authentication across every transport
- Rate limiting with 3 strategies (fixed window, sliding window, token bucket), a circuit breaker and caching
- LangChain/LangGraph integration with Python client support
- Published as NuGet packages through a three-stage pipeline with analysers, warnings as errors and a dependency scan
- Discontinued once the official SDK covered the same ground, with a documented migration path for the servers built on it

[.NET 9](#) [C#](#) [ASP.NET Core](#) [gRPC](#) [SignalR](#) [Polly](#)

AI-Powered Azure DevOps Integration via MCP

MCP server exposing Azure DevOps operations as AI-callable tools. Enables LLM agents to manage work items, pull requests, repositories, builds, tests, wikis and search through natural language. Rebuilt on .NET 10 and the official MCP C# SDK.

- 74 typed tools over streamable HTTP, covering work items, queries, pull requests, repositories, commits, builds, tests, wikis, search, teams and security groups
- Authentication scoped to the MCP session
- Retries apply only to reads, so a create is never sent twice
- Build monitoring through a job handle and poll pattern with exponential backoff and a circuit breaker
- Layered design (tools, managers, engines, accessors) enforced by convention tests
- Four quality gates, including a build that fails below 80% line coverage
- A generated tool catalogue, with tests that fail the build if the documentation and the server disagree

[.NET 10](#) [C#](#) [MCP C# SDK](#) [Azure DevOps REST API](#) [Polly](#)

MCP Server Suite for Collaboration, Data and Research

A family of MCP servers that give AI agents typed, authenticated access to everyday platforms: Microsoft Teams and Microsoft To Do through Microsoft Graph, Databricks, Toggl Track and the open web. They share layered architecture, human approval for writes, and a set of sign-in libraries for Microsoft Entra.

- Microsoft Teams: about 70 tools for reading chats and channels, approved sends, scheduling and audio transcription, in six layers enforced at compile time
- Microsoft To Do: delegated sign-in and name-based list resolution, with 116 offline tests and an 80% coverage gate
- Databricks: roughly 171 tools across 13 areas, split into read, write and admin servers so access is granted by which binary is deployed
- Toggl Track: 79 tools covering timers, projects, tasks, reports and insights
- Web research: several search providers queried in parallel and merged by reciprocal rank fusion, with an SSRF guard and a site allow-list
- Writes need human approval through MCP elicitation, so the model cannot approve its own actions, and third-party text is fenced as untrusted
- Microsoft Entra sign-in libraries: 10 credential strategies and a token refresh service that coalesces concurrent renewals

[.NET 10](#) [C#](#) [MCP C# SDK](#) [Microsoft Graph](#) [Microsoft Entra](#) [SQLite](#)

Governed AI Gateway

A .NET 10 gateway that puts chat, embeddings, image generation, speech, document reading, OCR and retrieval behind one API. It handles identity, role-based permissions, per-application budgets, usage metering and settlement, and a request audit in one place. Logan designed and built the production service, following a proof of concept by colleagues.

- Six iDesign layers enforced by the compiler, with one test project per source project
- Stored-procedure-only data layer: the application login has execute rights only, and a convention test guards the single executor
- Governance pipeline that checks permission and then budget before any model call

- Exactly-once settlement of streamed responses, with one code path serving both streamed and buffered calls
- Failover across named model connections, and retries limited to idempotent calls
- A job queue with a background worker for deferred work, authorised at submission and settled when it runs
- Audit redaction by field name, model retirement dates, and OpenAI-compatible passthrough routes
- 80% line coverage per layer with warnings as errors

[.NET 10](#) [C#](#) [ASP.NET Core](#) [SQL Server](#) [OpenTelemetry](#) [Kubernetes](#)

Hierarchical Code and Documentation Review Agents

A multi-agent system for automated review of pull requests. It runs specialised reviewer agents in parallel over a diff, has validators adjudicate, corroborates findings with static analysis, deduplicates, and posts inline comments or a report. A sibling system reviews documentation against the source code it describes.

- Three review intensities, running 9, 27 or 37 specialised reviewers in parallel
- 77 agent specifications, with 42 ecosystem pattern documents across 7 languages
- Backend coverage: C#, Java, Python, Go, Node.js, Ruby, Rust, PHP. Frontend coverage: React, Vue, Angular, Svelte
- CI/CD reviewers covering all ten OWASP Top 10 CI/CD Security Risks
- Six-step pipeline: trigger, review, decide, verify, deduplicate, publish, with validators iterating to convergence
- Verification through static analysis and reachability into a composite confidence score
- Documentation reviewers across four dimensions: standards compliance, fidelity to the code, clarity and diagram quality
- Model profiles (fast, standard, deep) that decouple agent specifications from vendor models
- 80% coverage gate with lint, security and specification checks in CI

[Python](#) [C#](#) [MCP](#) [A2A](#) [BM25 RAG](#) [Docker](#) [iDesign](#)

Agent Builder: An Agent That Builds Agents

An autonomous service that turns a natural-language specification into a standalone, containerised Python agent project. An LLM reasons through a typed twelve-phase pipeline while a .NET MCP server owns the file system, templates, validators and guards. The output is a runnable service with deployment files and a smoke test.

- 12 phases, 27 typed MCP tools, 4 guard layers and 6 archetype tiers
- 838 tests at 98% coverage
- Each phase is its own composed agent with a narrowed tool catalogue and an iteration budget
- The model supplies typed structured output and the server renders the source, so it never free-writes code that has to compile
- Phase executors verify completion claims against validators and make retries idempotent
- Generates chat agents, RAG agents, tool-using agents with a companion MCP server, and orchestrator hierarchies three levels deep

[Python](#) [FastAPI](#) [.NET](#) [C#](#) [MCP](#) [Docker](#) [JSON Schema](#)

Terminal AI Coding Assistant

A full-screen terminal coding assistant with tool use, a permission gate for destructive operations, persistent memory, planning, and session save, resume and rewind. It registers MCP tool servers, delegates to other agents over A2A, and also runs headless behind a VS Code extension.

- 10 built-in tools for files, shell, search, git and the web, plus dynamically loaded MCP tools
- Permission gate with allow, deny, always and never decisions and a diff preview
- 24 slash commands, session rewind and persistent project memory
- Agent-to-agent delegation as both client and server
- Two composition roots (terminal app and headless host), each with a test that builds the whole object graph with no network
- An SSRF guard that refuses private addresses by default

Python prompt_toolkit MCP A2A VS Code extension .NET

Planner and Proof Gate for AI Agents

A planning and verification service for other agents. A calling agent asks for a plan, does a step, and submits its work as a patch. The service rebuilds the workspace in its own sandbox, re-runs the acceptance checks, including hidden ones, and decides whether the step is verified. No API call lets the caller mark its own work done.

- Ten agents: a supervisor with a state machine, a planning orchestrator running four lenses as a declared graph, a plan reviewer, a proof adjudicator and a blind completion auditor
- Verification is reachable only through the gate, an invariant asserted by a test
- Hidden holdout checks that never leave the service
- Plans bound to the principal that owns them, with correlation and provenance identifiers
- Lessons from failed steps recorded for later plans

Python A2A .NET 10 MCP C# SDK

AI Use-Case Triage and SDLC Document Pipeline

Two connected multi-agent systems. The first assesses a proposed AI use case for data sensitivity, reuse of what already exists, and build effort, and returns one grounded verdict with a written proposal. The second turns an approved verdict into a full SDLC document set, and refuses to render any statement it cannot trace to a source.

- Triage: 19 agents on one shared foundation, with three orchestrators and their specialists
- Orchestration is deterministic code, and LLM judgement lives only in the specialists and the synthesiser
- Sequencing rules are encoded and not prompted: gate on sensitivity first, reuse before build, parallel fan-out where steps are independent
- A golden-set regression suite of ideas with known verdicts, and refusal on low confidence
- Document pipeline: 15 agents producing six documents, each ending with an addendum that maps every statement to its source
- An MCP server that renders Word documents by filling an existing template through Open XML, with tests that pin the template's structure

Python A2A MCP FAISS Open XML .NET 10 Docker

Automated CI/CD Control Review at Estate Scale

An automated review of every CI/CD pipeline in a large Azure DevOps estate against a fixed set of controls. It reads build definitions, pipeline YAML, branch policies and build timelines through the REST API, and writes one report per repository with roll-ups for each project and for the estate.

- 49 projects and 1,081 repositories reviewed against 17 fixed checks
- Shared pipeline templates resolved across repository boundaries, with conditions evaluated
- Every finding attributed to its origin: a shared template, a repository override, or the repository itself
- Per-repository reports, project and estate roll-ups, and registers of pipelines and service connections
- A steering committee briefing with charts, and a findings workbook for remediation planning

Azure DevOps REST API YAML pipelines Markdown Excel

Agentic Knowledge Ingestion and Curation Pipeline

An agentic service that keeps the knowledge behind AI assistants current. On a schedule it pulls changes from configured sources and hands them to a team of authoring agents, coordinated by an orchestrator agent over the A2A protocol, that write a readable corpus. The result passes a fail-closed sensitive-data gate and is then chunked, embedded and indexed. The readable text is published as a pull request for human review, and the assistant is told to reload. Logan architected and designed the service, and a colleague is building it on the agent framework.

- Architecture and design by Logan, with the build led by a colleague
- A new assistant is purely a configuration file
- Four source connector types behind one interface

- Agentic by design: an orchestrator runs every mechanical stage in process and calls four single-purpose agents over A2A, fanning out per document
- Agents are definitions and not code: a card, a system prompt and a template in a folder, served by one generic host
- Reviewable text and the built index go to different places, tied together by one committed pointer
- The index carries a fingerprint of its recipe, and an assistant refuses one that does not match
- An unchanged source is never re-authored, which keeps review free of churn
- A fail-closed sensitive-data gate before anything is indexed
- Tiered PDF extraction that falls back to OCR
- Index files verified by hash and signature before loading, and a circuit breaker per dependency

Python A2A Multi-Agent RAG FAISS OCR Config as Code

AI-Powered Knowledge Base System

Production knowledge management platform with hierarchical document organization, semantic search via vector embeddings, dynamic Qdrant container management, and conversational RAG interface. Includes admin dashboard, file upload, and audit logging.

- Hierarchical document organization with tagging
- Semantic search via vector embeddings
- Dynamic Qdrant container management
- Conversational RAG interface for natural language queries
- Admin dashboard with file upload and audit logging

.NET 9 Python React Qdrant FastAPI

Technical Skills

C# / .NET 10 (Primary) — High-performance services, MCP servers and frameworks, AI gateways, enterprise backend systems

TypeScript / React (Strong) — Modern frontends, real-time chat interfaces, admin dashboards

SQL (Strong) — T-SQL, complex schemas, query optimization

C++ (Experienced) — Performance-critical and security applications

Python (Primary) — LangChain, LangGraph, FastAPI, Pydantic, AI/ML pipelines

Dart / Flutter (Strong) — Cross-platform mobile and desktop applications

C (Strong) — Native terminal applications with ncurses, SQLite and multi-threaded networking

Frameworks & Tools

- LangChain and LangGraph for LLM orchestration
- Model Context Protocol (MCP) for AI agent tooling, on the official C# SDK and custom frameworks
- Agent-to-agent (A2A) protocol for multi-agent systems
- RAG pipelines with 14 chunking strategies, hybrid retrieval (BM25 + vector), RAPTOR, HyDE, ColBERT, Graph RAG, deep search, and hallucination detection
- Vector databases: Qdrant, FAISS, Pinecone, ChromaDB, Weaviate
- Multi-agent orchestration, recursive agent systems, and proof-gated agent verification
- Human-in-the-loop agent design: approval gates, two-step confirmation, untrusted text fencing
- AI security: multi-turn attack scoring, risk accumulation/decay, canary tokens, spotlighting, tool chain validation, capability restriction
- Context management: rolling window, fallback summarization, conversation reranking and compression
- Prompt engineering, prompt security, and semantic search
- Multi-protocol backend (gRPC, SignalR, SSE, Streamable HTTP, WebSocket, REST)
- Microsoft Graph and Microsoft Entra ID integration
- ncurses and SQLite for native terminal applications

- React and TypeScript for web frontends
- Flutter for cross-platform mobile/desktop
- Tailwind CSS for styling
- Docker, Docker Compose, Rancher, and Kubernetes for containerization
- Cross-platform deployment (Linux, Mac, Windows)
- Azure cloud infrastructure
- CI/CD pipelines
- NUnit, xUnit, pytest and Vitest for testing, with coverage gates in CI
- OpenTelemetry + Elasticsearch for observability
- A/B testing frameworks and experiment management

Architectural Concepts

- iDesign Methodology (trained by Juval Lowy)
- Domain-Driven Design (DDD)
- Clean Architecture
- Microservices and Event-Driven Architecture
- Authentication and Security (JWT, API keys, Azure AD, RBAC)
- Agentic AI Design Patterns (spotlighting, guardrails, tool use governance, human-in-the-loop orchestration, structured output filtering)
- Multi-turn security (prompt injection scanning, multi-turn attack scoring, canary tokens, risk accumulation/decay, capability restriction, output leakage detection)
- Content style enforcement and structured output generation with schema validation
- Database and Vector Storage (SQL Server, PostgreSQL, SQLite, vector databases)
- Observability patterns (correlation IDs, structured logging, OpenTelemetry)

Education

Diploma in C++ Programming

Intec College

Average of 73.1% - highest score in institution history as of 2006

Subjects: Intro to Programming, C++, Advanced C++, Systems Analysis and Design, Management Information Systems

Grade 12 (Matric with Merit)

Goudrif High School

Computer Training Institute floating trophy for best student (2002). Completed entire Matric Computer Studies syllabus in Grade 10.

Personal Interests

- Passionate about solving hard engineering problems
- Builds open source terminal software: tawk, a WhatsApp client for the terminal (<https://github.com/loganventer/tawk>), tawk-mcp, its MCP server (<https://github.com/loganventer/tawk-mcp>), and matrix-clock, a digital rain clock
- Built a mobile game with 8,000 downloads on Windows Phone Store
- Built an Eskom load-shedding monitoring app (1,200 downloads in 2 weeks)
- Built a TV backlight system with color perception based on pixel analysis
- Guitar player and vocalist with public performance experience
- Original songs played on Jacaranda radio
- Recorded an album
- Songwriting as a creative outlet

Core Values

- Clean architecture and deliberate design decisions
- Deterministic systems with well-defined failure boundaries
- Code that communicates intent
- Direct feedback and honest technical discourse

References

Derivco

Chris Coetzee: +27 82 548 8817
Ockert Du Preez: +27 72 829 0170
Paul Kotze: +27 82 901 7953
Albie van Zyl: +27 79 492 5393
Gopi Kistasami: +27 83 616 1371

Seecrypt

Annalinde Singh: +27 76 640 4766
Dino Fizzotti: +44 734 041 5199

Infotech / Enermatics

Melissa Wittstock: +27 82 750 0065

City of Tshwane

Martie J. Matthee: +27 12 358 8269

Books-u-profit

Dries Venter: +27 12 800 3327

Dynamic Visual Technologies

Khumo Jele: +27 81 447 9732

UD Trucks / Covariant Consulting

Ronete McKenzie: +27 61 682 2350

Information Trade World

Maria Jacobs: +27 74 254 6027

Marnet Signs

Martie Roberts / Eddie Bekker: +27 12 734 4990

Nexor Signs

Dries du Plessis: +27 12 333 3384